



HUST

ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.



ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

FUNDAMENTALS OF OPTIMIZATION

Heuristic Algorithm

ONE LOVE. ONE FUTURE.

Heuristic algorithm

- Heuristic algorithms are strategies designed to efficiently tackle complex optimization problems by providing approximate solutions when exact methods are impractical.
- This is achieved by trading optimality, completeness, accuracy, or precision for speed.
- The objective of a heuristic is to produce a solution in a reasonable time frame that is good enough for solving the problem at hand. This solution may not be the best of all the solutions to this problem, or it may simply approximate the exact solution. But it is still valuable because finding it does not require a prohibitively long time.

- **Greedy Algorithm**
 - Introduction to greedy algorithm
 - Example 1: Fractional Knapsack
 - Example 2: 0/1 Knapsack
 - Example 3: Multi-Knapsack
 - Example 4: TSP
 - Example 5: Set cover problem
 - Exercises
- Local Search

Greedy algorithm

Greedy algorithms are a class of algorithms that make **locally optimal** choices at each step with the hope of finding a **global optimum** solution.

- **S**: a solution is represented by a set of components
- **C**: set of candidates of components to be added to the solution
- **select(C)**: select the most promising component among candidates
- **solution(S)**: return true if **S** is a solution to the original problem
- **feasible(S)**: return true if **S** does not violate any constraints

```
Greedy() {  
    S = {};  
    while C  $\neq$   $\emptyset$  and  
        not solution(S){  
        x = select(C);  
        C = C \ {x};  
        if feasible(S  $\cup$  {x}) {  
            S = S  $\cup$  {x};  
        }  
    }  
    return S;  
}
```

- **Greedy Algorithm**
 - Introduction to greedy algorithm
 - **Example 1: Fractional Knapsack**
 - Example 2: 0/1 Knapsack
 - Example 3: Multi-Knapsack
 - Example 4: TSP
 - Example 5: Set cover problem
 - Exercises
- Local Search

Fractional knapsack

Problem: There are now a set of items where each item has its value and weight. You have a knapsack and hope to put in as many valuable items as possible. However, the knapsack's capacity is limited to some weight W that the total weight of selected items should not exceed W . The goal is to select a collection of items achieving the maximum total value while satisfying the capacity restriction. This problem in general is called the knapsack problem and it has many variations. In the fractional setting here, each item could be arbitrarily divided so that only a fraction of the item is taken. For example, suppose item A has weight 4 and value 10. Then a $1/2$ fraction of item A will occupy 2 weight and has value 5.

Example:

- **Input:** value = [60, 100, 120], weight = [10, 20, 30], weight capacity = 50
- **Output:** 240
- **Explanation:** By taking items of weight 10 and 20 kg and $2/3$ fraction of 30 kg. Total price will be $60+100+(2/3)(120) = 240$

Fractional knapsack

- Idea: Sort items in decreasing order based on the ratio of their value to their weight

Fractional knapsack

```
def fractionalKnapsack(val, wt, capacity):
    n = len(val)

    # Create list to store value and weight
    # items[i][0] = value, items[i][1] = weight
    items = [[val[i], wt[i]] for i in range(n)]

    # Sort items based on value-to-weight ratio in descending order
    items.sort(key=lambda x: x[0]/x[1], reverse=True)

    res = 0.0
    currentCapacity = capacity

    # Process items in sorted order
    for i in range(n):

        # If we can take the entire item
        if items[i][1] <= currentCapacity:
            res += items[i][0]
            currentCapacity -= items[i][1]
```

```
        # Otherwise take a fraction of the item
        else:
            res += (1.0 * items[i][0] / items[i][1]) * currentCapacity

            # Knapsack is full
            break

    return res

if __name__ == "__main__":
    val = [60, 100, 120]
    wt = [10, 20, 30]
    capacity = 50

    print(fractionalKnapsack(val, wt, capacity))
```

- **Greedy Algorithm**

- Introduction to greedy algorithm
- Example 1: Fractional Knapsack
- **Example 2: 0/1 Knapsack**
- Example 3: Multi-Knapsack
- Example 4: TSP
- Example 5: Set cover problem
- Exercises

- Local Search

- **Problem:** Given n items where each item has some weight and profit associated with it and given a bag with capacity W , [i.e., the bag can hold at most W weight in it]. The task is to put the items into the bag such that the sum of profits associated with them is the maximum possible.
- **Example:**
 - **Input:** $W = 4$, $\text{profit}[] = [1, 2, 3]$, $\text{weight}[] = [4, 5, 1]$
 - **Output:** 3
 - **Explanation:** There are two items which have weight less than or equal to 4. If we select the item with weight 4, the possible profit is 1. And if we select the item with weight 1, the possible profit is 3. So, the maximum possible profit is 3. Note that we cannot put both the items with weight 4 and 1 together as the capacity of the bag is 4.

- **Idea 1:** Sort items in decreasing order based on their value
- **Idea 2:** Sort items in increasing order based on their weight
- **Idea 3:** Sort items in decreasing order based on the ratio of their value to their weight
- **Idea 4:** The best one among the solutions returned from above ideas

0/1 Knapsack

```
def Knapsack(val, wt, capacity, idea):
    n = len(val)

    # Create list to store value and weight
    # items[i][0] = value, items[i][1] = weight
    items = [[val[i], wt[i]] for i in range(n)]
    # Sort items based on value-to-weight ratio in descending order
    if idea == 1:
        items.sort(key=lambda x: x[0], reverse=True)
    elif idea == 2:
        items.sort(key=lambda x: x[1], reverse=False)
    else:
        items.sort(key=lambda x: x[0]/x[1], reverse=True)

    res = 0.0
    currentCapacity = capacity
    # Process items in sorted order
    for i in range(n):
        # If we can take the entire item
        if items[i][1] <= currentCapacity:
            res += items[i][0]
            currentCapacity -= items[i][1]
        # Otherwise take a fraction of the item
        else:
            # Knapsack is full
            break
    return res
```

```
if __name__ == "__main__":
    val = [60, 100, 120]
    wt = [10, 20, 30]
    capacity = 50

    sol1 = Knapsack(val, wt, capacity, 1)
    sol2 = Knapsack(val, wt, capacity, 2)
    sol3 = Knapsack(val, wt, capacity, 3)
    sol4 = max(max(sol1, sol2), sol3)
    print(sol1, sol2, sol3, sol4)
```

- **Greedy Algorithm**
 - Introduction to greedy algorithm
 - Example 1: Fractional Knapsack
 - Example 2: 0/1 Knapsack
 - **Example 3: Multi-Knapsack**
 - Example 4: TSP
 - Example 5: Set cover problem
 - Exercises
- Local Search

Multi-Knapsack problem

- **Problem:** Given unlimited number of bins having capacity Q and n items $1, 2, \dots, n$ in which the weight of item i is $w(i)$. How to put these n items into bins such that the total weight of items put into each bin cannot exceed Q and the number of bins used is minimal.

Multi-Knapsack problem

```
greedyMNS(){
    binNum = 0
    for i = 1 to n do{
        b = select(load,w(i));
        if b = NULL then {
            binNum = binNum + 1;
            b = binNum;
            load[b] = 0;
        }
        binOfItem[i] = b;
        load[b] = load[b] + w[i];
    }
    return binOfItem;
}
```

```
selectFirstFitBin(load,w){
    for b = 1 to binNum do {
        if load[b] + w <= Q then
            return b;
    }
    return NULL;
}
```

```
selectBestFitBin(load,w){
    R = -∞; sel_b = NULL
    for b = 1 to binNum do {
        if load[b] + w <= Q and (R < Q-load[b]-w) then
            R = Q - load[b] - w; sel_b = b;
    }
    return sel_b;
}
```

Multi-Knapsack problem

```
greedyMNS(){
    binNum = 0
    sort items in an decreasing order of
    weights; (suppose  $w(1) \geq \dots \geq w(n)$ )
    for i = 1 to n do{
        b = select(load,w(i));
        if b = NULL then {
            binNum = binNum + 1;
            b = binNum;
            load[b] = 0;
        }
        binOfItem[i] = b;
        load[b] = load[b] + w[i];
    }
    return binOfItem;
}
```

```
selectFirstFitBin(load,w){
    for b = 1 to binNum do {
        if load[b] + w <= Q then
            return b;
    }
    return NULL;
}
```

```
selectBestFitBin(load,w){
    R =  $-\infty$ ; sel_b = NULL
    for b = 1 to binNum do {
        if load[b] + w <= Q and (R < Q-load[b]-w) then
            R = Q - load[b] - w; sel_b = b;
    }
    return sel_b;
}
```

Multi-Knapsack problem

```
greedyMNS(){
    binNum = 0
    candidates = {1,...,n}
    while (candidates not empty){
        R =  $\infty$ ; sel_bin = NULL; sel_item = 0;
        for i in candidates do{
            for b = 1 to binNum do{
                if load[b] + w(i)  $\leq$  Q and (Q - load[b] - w(i) < R then{
                    R = Q - load[b] - w(i); sel_bin = b; sel_item = i;
                }
            }
        }
        if (sel_bin = NULL){
            binNum = binNum + 1
        }else{
            candidates.remove(sel_item);
            binOfItem[sel_item] = sel_bin; load[sel_bin] = load[sel_bin] + w(sel_item);
        }
    }
    return binOfItem;
}
```

- **Greedy Algorithm**

- Introduction to greedy algorithm
- Example 1: Fractional Knapsack
- Example 2: 0/1 Knapsack
- Example 3: Multi-Knapsack
- **Example 4: TSP**
- Example 5: Set cover problem
- Exercises

- Local Search

- **Problem:** Given n points $1, 2, \dots, n$ in which $d(i, j)$ is the distance from point i to point j . Find the shortest closed tour starting from 1 visiting other points and terminating at 1 such that the total travel distance is minimal.

- Idea 1:
 - The tour is initialized by point 1
 - At each step, select the nearest point to the last point of the tour under construction and add this point to the end of the tour
- Idea 2:
 - Apply idea 1 with different starting point
 - Select the shortest tour
- Idea 3:
 - Build a minimum spanning tree T for the graph G
 - Perform DFS traverse tree T so that a visit sequence of points
- Idea 4: Shortest one among three solutions provided by above ideas

- **Greedy Algorithm**

- Introduction to greedy algorithm
- Example 1: Fractional Knapsack
- Example 2: 0/1 Knapsack
- Example 3: Multi-Knapsack
- Example 4: TSP
- **Example 5: Set cover problem**
- Exercises

- Local Search

Set cover problem

- **Problem:** Suppose we are going to form a team to accomplish a mission. Team members are selected from a group of candidates each of whom specializes in several skills (such as reconnaissance, combat, medical support, etc.). The objective is to make the team as small as possible while satisfying that every required skill is owned by at least one team member. And this is one situation that needs an algorithm for set cover problem.

Set cover problem

- **Idea:** Select members with the largest set of needed skills.

- **Greedy Algorithm**

- Introduction to greedy algorithm
- Example 1: Fractional Knapsack
- Example 2: 0/1 Knapsack
- Example 3: Multi-Knapsack
- Example 4: TSP
- Example 5: Set cover problem
- **Exercises**

- **Local Search**

Balanced Class Teacher Assignment

- There are n classes labeled $1, 2, \dots, n$ that have already been scheduled in a timetable, and they need to be assigned to m teachers labeled $1, 2, \dots, m$
- Each class i has $T(i)$, a list of teachers who can teach it ($i = \{1, \dots, n\}$), and $c(i)$, the number of credits for the subject of that class.
- Since the timetable has been pre-arranged, there exists a set Q of pairs of classes (i, j) that are scheduled at the same time (these two classes cannot be assigned to the same teacher).
- Find an assignment of classes to teachers such that the maximum total number of credits assigned to any one teacher is minimized.

Balanced Class Teacher Assignment

- Example

Class	0	1	2	3	4	5	6	7	8	9	10	11	12
Credit	3	3	4	3	4	3	3	3	4	3	3	4	4

Teacher	List of classes that the teach can teach
0	0, 2, 3, 4, 8, 10
1	0, 1, 3, 5, 6, 7, 8
2	1, 2, 3, 7, 9, 11, 12

List Q

0	2
0	4
0	8
1	4
1	10
3	7
3	9
5	11
5	12
6	8
6	12

Balanced Class Teacher Assignment

- Example

Class	0	1	2	3	4	5	6	7	8	9	10	11	12
Credit	3	3	4	3	4	3	3	3	4	3	3	4	4

Teacher	List of classes that the teach can teach
0	0, 2, 3, 4, 8, 10
1	0, 1, 3, 5, 6, 7, 8
2	1, 2, 3, 7, 9, 11, 12

Assignment
solution



Teacher	List of classes assigned to the teacher	Total credits
0	2, 4, 8, 10	15
1	0, 1, 3, 5, 6	15
2	7, 9, 11, 12	14

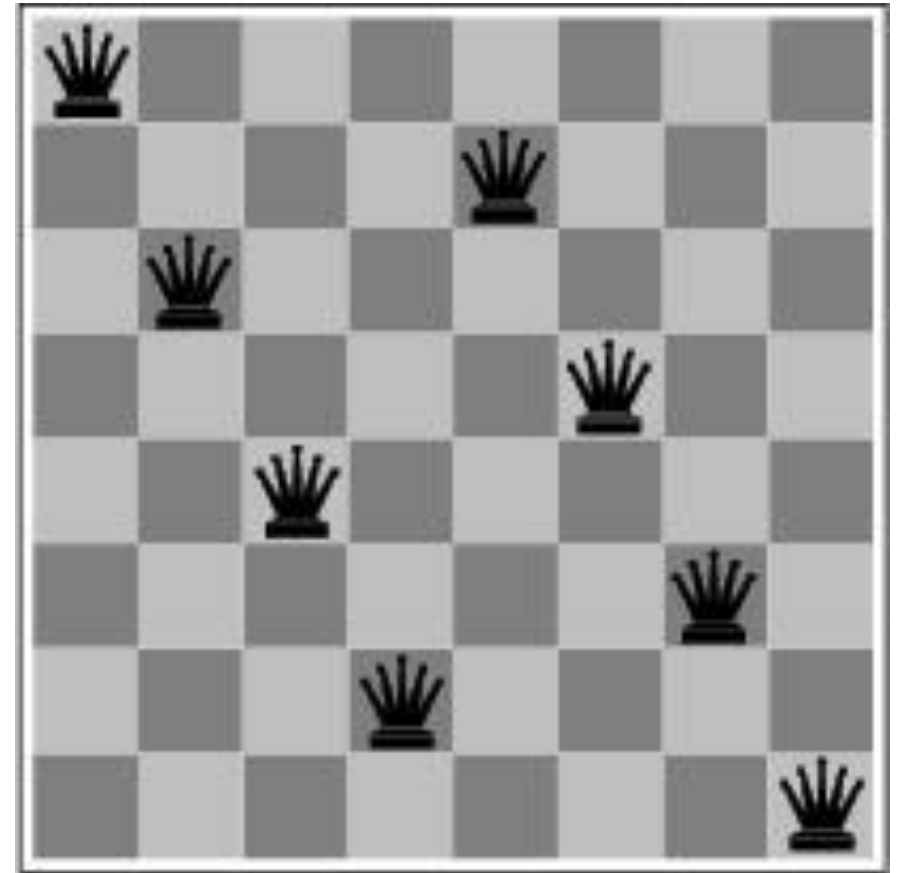
List Q

0	2
0	4
0	8
1	4
1	10
3	7
3	9
5	11
5	12
6	8
6	12

- Greedy Algorithm
- **Local Search**
 - Introduction to LS
 - Example 1: LS algorithm for Two-Partition Problem
 - Example 2: N-Queen
 - Example 3: TSP

Local search and optimization

- This lecture: a state is solution to problem
 - E.g., 8-queens
- Different algorithms can be used
 - Depth First Branch and Bound
 - Local search



Local search and optimization

Goal Satisfaction

reach the goal node
Constraint satisfaction

Optimization

optimize(objective fn)
Constraint Optimization

You can go back and forth between the two problems
Typically in the same complexity class

Local search and optimization

- Local search
 - Keep track of single current state
 - Move only to neighboring states
 - Ignore paths
- Advantages:
 - Use very little memory
 - Can often find reasonable solutions in large or infinite (continuous) state spaces.
- “Pure optimization” problems
 - All states have an objective function
 - Goal is to find state with max (or min) objective value
 - Local search can do quite well on these problems.

- Random Sampling
 - Generate a state randomly
- Random Walk
 - Randomly pick a neighbor of the current state

Hill-climbing (Greedy Local Search) - max version

function HILL-CLIMBING(*problem*) **return** a state that is a local maximum

input: *problem*, a problem

local variables: *current*, a node.

neighbor, a node.

current $\leftarrow$ MAKE-NODE(INITIAL-STATE[*problem*])

loop do

neighbor $\leftarrow$ a highest valued successor of *current*

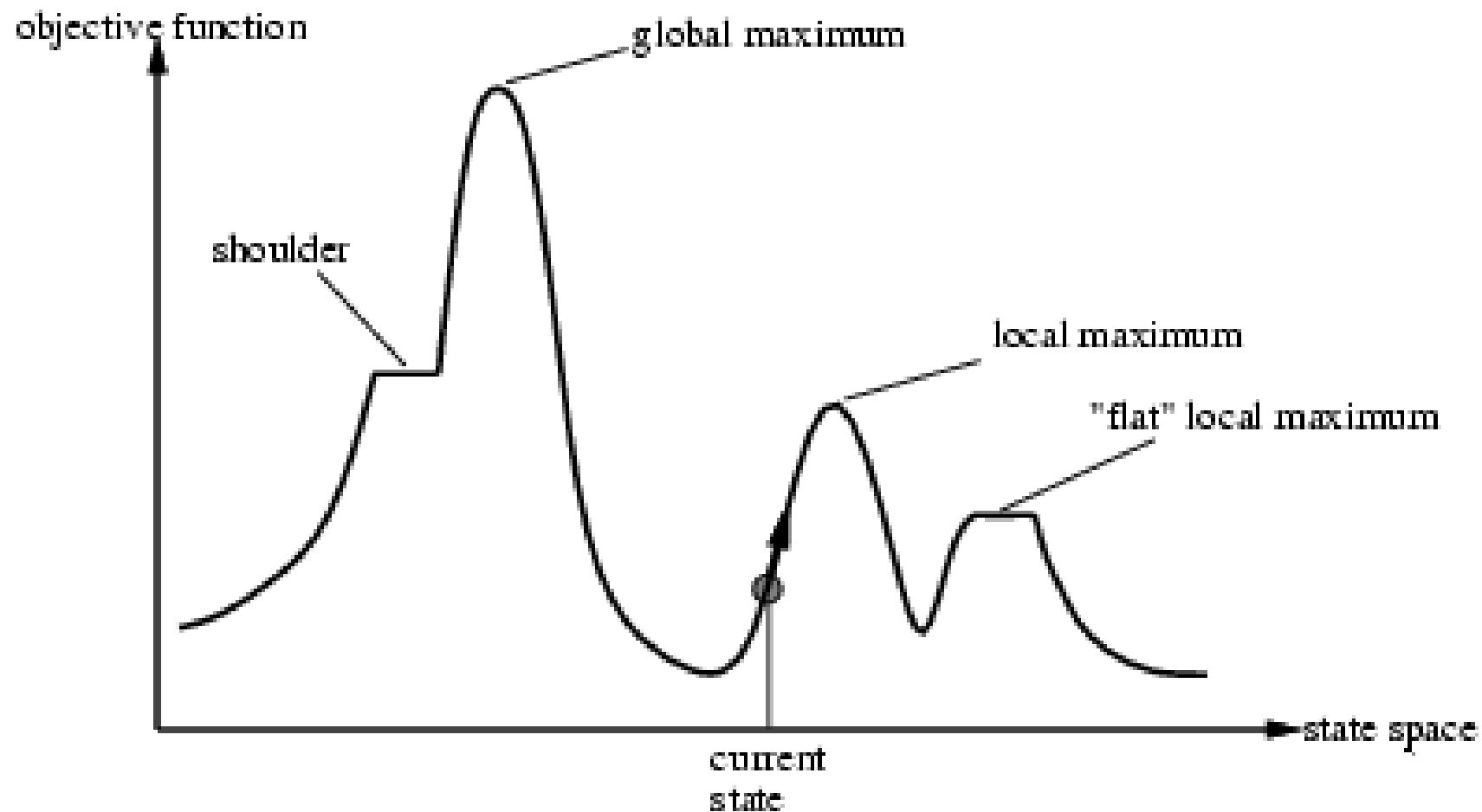
if VALUE [*neighbor*] $\leq$ VALUE[*current*] **then return** STATE[*current*]

current $\leftarrow$ *neighbor*

Hill-climbing search

- “a loop that continuously moves towards increasing value”
 - terminates when a peak is reached
- Value can be either
 - Objective function value
 - Heuristic function value (minimized)
- Hill climbing does not look ahead of the immediate neighbors
- Can randomly choose among the set of best successors
 - if multiple have the best value
- “climbing Mount Everest in a thick fog with amnesia”

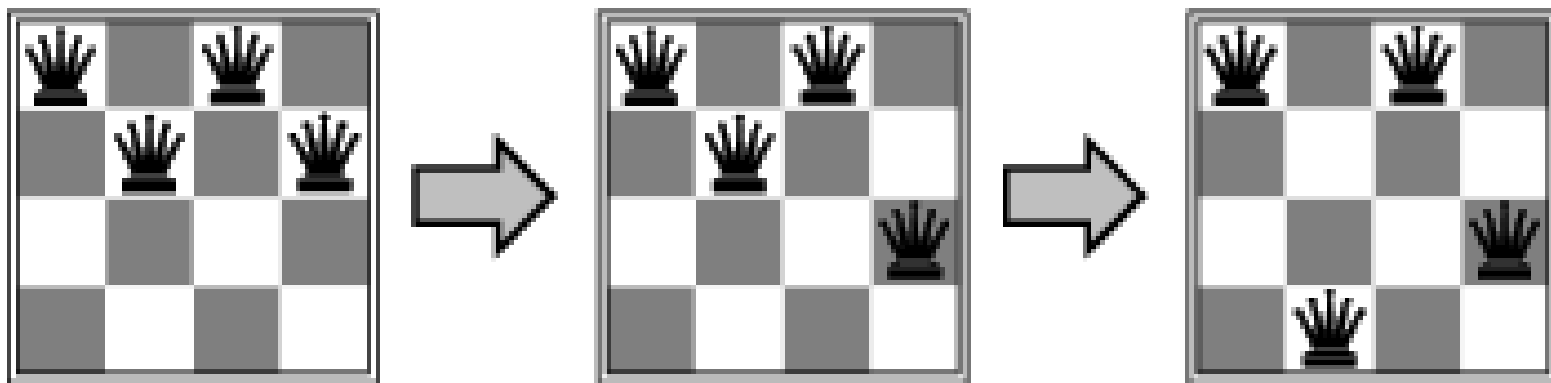
“Landscape” of search



Hill Climbing gets stuck in local minima depending on?

Example: n -queens

- Put n queens on an $n \times n$ board with no two queens on the same row, column, or diagonal



- Is it a satisfaction problem or optimization?

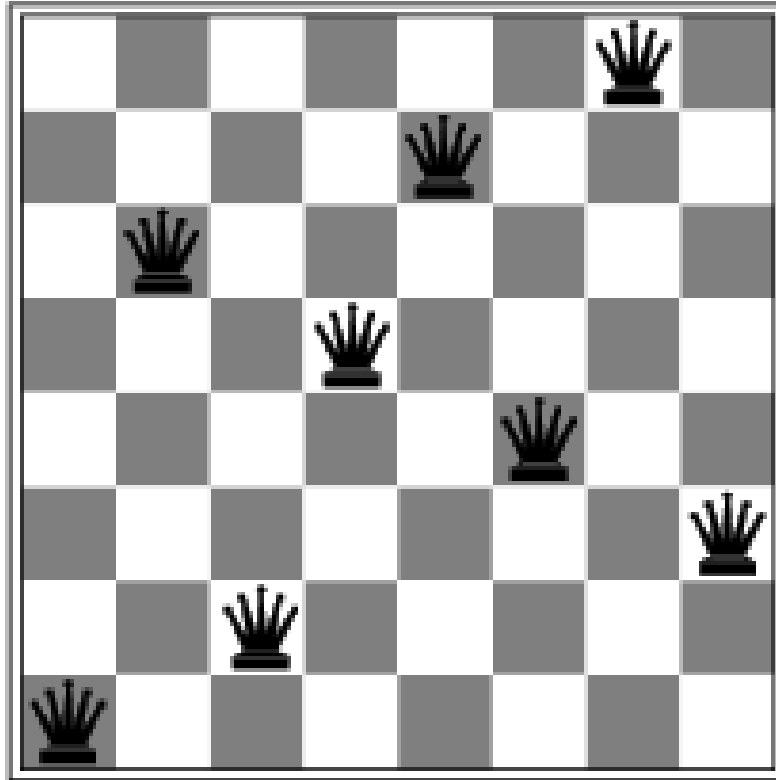
Hill-climbing search: 8-queens problem

18	12	14	13	13	12	14	14
14	16	13	15	12	14	12	16
14	12	18	13	15	12	14	14
15	14	14	♚	13	16	13	16
♚	14	17	15	♚	14	16	16
17	♚	16	18	15	♚	15	♚
18	14	♚	15	15	14	♚	16
14	14	13	17	12	14	12	18

- Need to convert to an optimization problem
- h = number of pairs of queens that are attacking each other
- $h = 17$ for the above state

- State
 - All 8 queens on the board in some configuration
- Successor function
 - move a single queen to another square in the same column.
- Example of a heuristic function $h(n)$:
 - the number of pairs of queens that are attacking each other
 - (so we want to minimize this)

Hill-climbing search: 8-queens problem



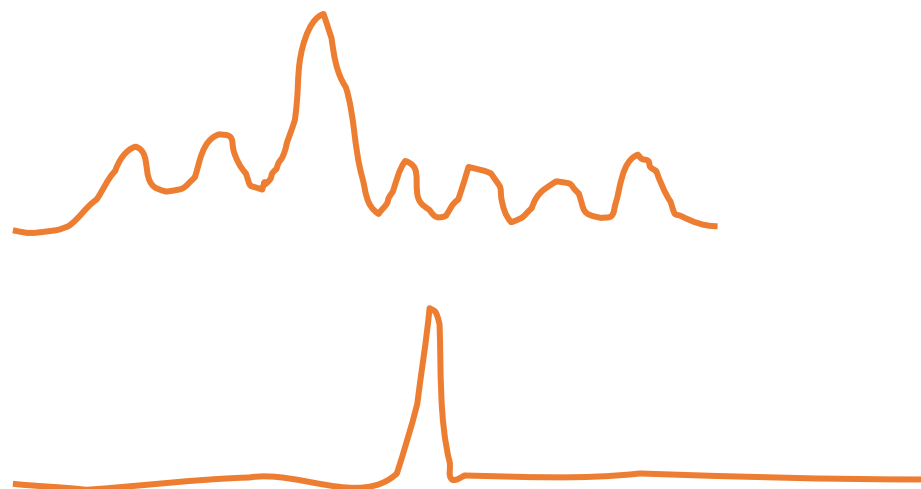
- Is this a solution?
- What is h ?

Hill-climbing on 8-queens

- Randomly generated 8-queens starting states...
- 14% the time it solves the problem
- 86% of the time it get stuck at a local minimum

Hill Climbing Drawbacks

- Local maxima
- Plateaus



Systematic heuristics (1)

- Best neighbor
 - Choosing the neighbor with the best evaluation
 - Scan all next states (neighbors)
- First neighbor
 - Select the first-found neighbor which is better than current solution
 - Might increase the number of iteration to reach local minima

- Multistage heuristics
 - Balance between the greedy aspect and the computation time
 - Choice of a neighbor can be considered as several decisions that are performed together in the best-improvement heuristics
 - Example: Choose a pair of queens to swap to each other in N-queen problem, in two-stage heuristic: it first selects the queen Q1 with largest violation, then chooses the best queen Q2 to swap with Q1.

Random walks (1)

- Random-walk heuristics are quite different to systematic heuristics: They select a neighbor randomly and decide whether to accept it as the next solution
- Random improvement
 - Simplest example of random-walk heuristics
 - Accept a neighbor if it improves the current solution

```
1. function S-RANDOMIMPROVEMENT(N,s)
2.   select  $n \in N$  with probability  $1/\#N$ ;
3.   if  $f(n) < f(s)$  then
4.     return  $n$ ;
5.   else
6.     return  $s$ ;
```

Random walks (2)

- The Metropolis heuristics
 - Extension of random improvement
 - Moves degrading the objective value is allowed with a small probability

```
1.  function S-METROPOLIS[T](N,s)
2.      select  $n \in N$  with probability  $1/\#N$ ;
3.      if  $f(n) \leq f(s)$  then
4.          return  $n$ ;
5.      else with probability  $\exp(\frac{-(f(n)-f(s))}{t})$ 
6.          return  $n$ ;
7.      else
8.          return  $s$ ;
```



Example 1: LS for Two-Partition Problem

- **Problem:** Whether a given a set S of n integers is partitioned into two disjoint subsets S_1 and S_2 such that the sum of integers in S_1 is equal to the sum of integers in S_2 ?
- LS algorithm design:
 - How to represent a feasible solution to the problem?
 - ➔ A binary vector of n elements
 - What are the neighbor solutions of a given solution or the neighborhood definitions?
 - ➔ Move an integer from one set to another set
 - ➔ Swap position of two integers from two sets

Example 2: LS for N-Queen

- **Problem:** Arrang n queens on a board of $n \times n$ such that no two queens can attack each other.
- LS algorithm design:
 - How to represent a feasible solution to the problem?
 - ➔ A vector $X = (x_1, \dots, x_n)$ of n decision variables, in which x_i is the row of the queen on the column i
 - What are the neighbor solutions of a given solution or the neighborhood definitions?
 - ➔ Move one queen
 - ➔ Swap positions of two queens

Example 3: TSP

- Problem: Finding a closed tour visiting each city once such that total distance is minimal
- LS algorithm design:

A large graphic on the left side of the slide. It features a dark blue background with a circular pattern of red dots of varying sizes, creating a sense of depth and movement. The word "HUST" is centered within this graphic in a bold, white, sans-serif font.

HUST

THANK YOU !